Software Engineering — Digital Assignment 3 / Review 3

PreMerge

Part A — Evidence Pack

Integration / regression / mutation testing, version management & system building, screenshots of developed functionality, and tools & technologies used.

Project team	
Vikram (`@Waterbottles792`)	Core engine, CLI, GitHub Action
Kanika Rathore (`@kanika10-hub`)	Documentation & planning
Repository: `github.com/Waterbottles792/PreMerge`	
Register No. / Name / Slot	[to be filled by student]

1. Integration, Regression and Mutation Testing

All screenshots below were captured by running the commands shown on the PreMerge source tree (Python 3.13, pytest 9.1, pytest-cov 7.1, mutmut 3.8). The suite builds throw-away git repositories, so every run is hermetic.

1.1 Integration testing

Eight end-to-end tests drive the real `premerge analyze` CLI (via click's `CliRunner`) against synthetic git repositories, exercising CLI → scoring → overlap → coupling → git_utils → report together: pair mode, all-pairs mode, `--base` mode, hidden-coupling detection, JSON/table rendering, custom config, and cold-vs-cached equivalence.

A1_integration_tests

```
$ python -m pytest tests/test_cli_integration.py -v --no-header -p no:cacheprovider
============================================== test session starts ==============================================
collecting ... collected 8 items

tests/test_cli_integration.py::test_cli_pair_json_reports_line_overlap PASSED                          [ 12%]
tests/test_cli_integration.py::test_cli_detects_hidden_coupling_without_file_overlap PASSED            [ 25%]
tests/test_cli_integration.py::test_cli_all_pairs_ranked_highest_risk_first PASSED                     [ 37%]
tests/test_cli_integration.py::test_cli_base_mode_compares_every_branch_to_base PASSED                 [ 50%]
tests/test_cli_integration.py::test_cli_table_output_renders_title_and_branches PASSED                 [ 62%]
tests/test_cli_integration.py::test_cli_no_other_branches_message PASSED                               [ 75%]
tests/test_cli_integration.py::test_cli_custom_config_changes_score PASSED                             [ 87%]
tests/test_cli_integration.py::test_cli_warm_cache_gives_same_result_as_cold PASSED                    [100%]

============================================== 8 passed in 7.55s ==============================================
```

Figure 1 — 8/8 CLI integration tests passing (tests/test_cli_integration.py)

1.2 Regression testing

The regression suite re-runs every unit and integration test on each change (locally and in GitHub Actions on Python 3.11 and 3.12). It pins previously untested behaviour: range-intersection boundaries, pure-deletion hunks, binary files, merge-commit exclusion, partial/missing config, JSON schema stability and the file-overlap-without-line-overlap score.

A2_regression_tests

```
$ python -m pytest tests/test_regression.py tests/test_coupling.py tests/test_overlap.py tests/test_scoring.py -v --no-header -p no:cacheprovider
============================================== test session starts ==============================================
collecting ... collected 14 items

tests/test_regression.py::test_ranges_intersect_boundaries PASSED                                      [  7%]
tests/test_regression.py::test_pure_deletion_hunk_counts_as_one_line PASSED                            [ 14%]
tests/test_regression.py::test_binary_file_is_file_level_only PASSED                                   [ 21%]
tests/test_regression.py::test_commit_log_skips_merge_commits PASSED                                   [ 28%]
tests/test_regression.py::test_config_defaults_partial_and_missing PASSED                              [ 35%]
tests/test_regression.py::test_json_report_schema PASSED                                               [ 42%]
tests/test_regression.py::test_file_overlap_without_line_overlap_scores_lower PASSED                   [ 50%]
tests/test_coupling.py::test_strong_coupling_between_cochanged_files PASSED                            [ 57%]
tests/test_coupling.py::test_no_coupling_for_unrelated_file PASSED                                     [ 64%]
tests/test_coupling.py::test_coupling_is_symmetric PASSED                                              [ 71%]
tests/test_coupling.py::test_cache_roundtrip PASSED                                                    [ 78%]
tests/test_overlap.py::test_line_level_overlap_detected PASSED                                         [ 85%]
tests/test_overlap.py::test_no_overlap_for_disjoint_files PASSED                                       [ 92%]
tests/test_scoring.py::test_overlapping_branches_score_higher_than_disjoint PASSED                     [100%]

============================================== 14 passed in 9.15s ==============================================
```

Figure 2 — Full regression run: unit + regression + integration tests

A3_coverage

```
$ python -m pytest -q --no-header -p no:cacheprovider --cov=premerge --cov-report=term-missing 2>&1 | tail -22
.....................                                                                                [100%]
================================================ tests coverage ================================================
_____________________________________ coverage: platform linux, python 3.13.16-final-0 _____________________________________

Name                     Stmts   Miss  Cover   Missing
------------------------------------------------------
premerge/__init__.py         1      0   100%
premerge/cli.py             41      3    93%   54-55, 79
premerge/config.py          18      0   100%
premerge/coupling.py        79      3    96%   46, 106, 124
premerge/git_utils.py       64      5    92%   25, 35, 85-86, 105
premerge/overlap.py         29      0   100%
premerge/report.py          30      6    80%   16-17, 28, 38-40
premerge/scoring.py         34      0   100%
------------------------------------------------------
TOTAL                      296     17    94%
22 passed in 15.87s
```

*Figure 3 — Coverage report: **22 passed, 94 % statement coverage** (was 7 tests / 67 % before this review)*

1.3 Mutation testing

Mutation testing was run with **mutmut** over all eight modules of `premerge/`. mutmut injects small faults (operator swaps, constant changes, string edits) and checks that the test suite fails; a surviving mutant marks a test gap.

Mutation testing (mutmut 3.8) - per-module score

```
$ python docs/review3/mutation_summary.py; echo; echo 'Sample of results:'; head -6 docs/review3/logs/mutmut_results.txt
module        killed  survived  no tests   score
config            37         4         0     90%
coupling          98        40         0     71%
git_utils        128        19        29     73%
overlap           53         8         0     87%
report            49        52         0     49%
scoring           60        10         0     86%
TOTAL            425       133        29     72%

Sample of results:
/usr/local/lib/python3.13/dist-packages/mutmut/configuration.py:143: UserWarning: The config paths_to_mutate is deprecated. Please rename it to source_paths
  warnings.warn("The config paths_to_mutate is deprecated. Please rename it to source_paths")
/usr/local/lib/python3.13/dist-packages/mutmut/configuration.py:152: UserWarning: The config tests_dir is deprecated. Please add the path to pytest_add_cli_args_test_selection instead
  warnings.warn(
    premerge.report.x_print_table__mutmut_1: killed
    premerge.report.x_print_table__mutmut_2: survived
```

Figure 4 — Mutation score per module: 425 of 587 mutants killed (72 %)

> **Interpretation.** The core algorithm modules score well — `config` 90 %, `overlap` 87 %, `scoring` 86 %. The weakest module is `report` (49 %): most survivors there mutate cosmetic Rich table strings, styles and column titles that tests intentionally do not pin. `git_utils` has 29 mutants in error-handling paths no test reaches. Surviving mutants are listed in `docs/review3/logs/mutmut_results.txt` and are the backlog for further tests.

2. Version Management and System Building

2.1 Version management (Git + GitHub, GitHub Flow)

The project uses **GitHub Flow**: a protected `main`, short-lived `feature/*` branches, pull requests with required checks, squash/merge and branch deletion. The history below shows the feature branches `feature/project-documentation` (PR #28) and `feature/docker-and-docs` (PR #29) merged into `main`.

Version management: commit graph

```
$ git log --graph --oneline --decorate --all -n 28 --color=always
* c1fd102 (HEAD -> claude/exciting-cray-blh904, origin/main, origin/claude/exciting-cray-blh904, main) Update summary in README.md
* 7307cc6 Revise README with software design and section updates
* 0c1e83b Add files via upload
* c647143 Add files via upload
* 0071f4c Create diagram.md
* c8d00ca Add files via upload
* a2208dd Create figma.md
* 2c89595 Create README.md
* e1ad0f2 Delete docs/design directory
* 75cbda3 Create links.md
* 705c88b Delete docs/Design directory
* 5a08579 Create links.md
* 907758c Update Kanika Rathore's GitHub link in README
* b4dc05b Update Kanika Rathore's GitHub handle in README
* e014b14 Fill in changelog for Kanika's docs PR and today's Docker/design work
*   cb8ad83 Merge pull request #29 from Waterbottles792/feature/docker-and-docs
|\
| * 56fc37d Add Docker support, complete architecture diagram, wire up Figma link
|/
*   31ce32d Merge pull request #28 from Waterbottles792/feature/project-documentation
|\
| * 91db73a Updated project files
| * 29d2a3c Restructure README and document GitHub Flow branching strategy
| * 3e9555a Add design and planning documentation
| * 2033a63 Add CI test workflow, PR template and contributing guide
| * 7112694 Expand .gitignore for build, coverage and editor artifacts
| * fe61cb4 Remove stray codename prefix from source comments
|/
* 6daa7ff Change header from H1 to H2 in README
* a361060 Attribute changelog entries by date and name
* b41bde7 Add changelog documenting today's initial build
* a991ab4 Remove conflict-demo scaffolding file
```

Figure 5 — git log --graph: feature branches merged through pull requests

Version management: branches, remote, contributors

```
$ git branch -a -vv --color=always | cut -c1-110; echo; git remote -v; echo; git shortlog -sn --all; echo; echo 'Total commits:' $(git rev-list --count origin/...
* claude/exciting-cray-blh904               c1fd102 Update summary in README.md
  main                                      c1fd102 [origin/main] Update summary in README.md
  remotes/origin/claude/exciting-cray-blh904 c1fd102 Update summary in README.md
  remotes/origin/main                       c1fd102 Update summary in README.md

origin  https://github.com/Waterbottles792/PreMerge (fetch)
origin  https://github.com/Waterbottles792/PreMerge (push)

    17  Waterbottles792
    16  Kanika Rathore
     6  kanika
     1  Waterbottle

Total commits: 40

cb8ad83 Merge pull request #29 from Waterbottles792/feature/docker-and-docs
31ce32d Merge pull request #28 from Waterbottles792/feature/project-documentation
b485884 Merge conflict-landed into main
```

Figure 6 — Branches, remote, contributors and merge commits

Version management: CI + PR template + changelog

```
$ ls .github .github/workflows; echo; head -20 .github/pull_request_template.md; echo ...; grep -n '^## ' CHANGELOG.md
.github:
pull_request_template.md
workflows

.github/workflows:
premerge.yml
tests.yml

## What does this PR do?

<!-- One or two sentences. What changes, and why. -->

## Related issue

<!-- Closes #123 — or "n/a" -->

## Type of change

- [ ] Feature (`feature/*`)
- [ ] Bug fix (`fix/*`)
- [ ] Documentation (`docs/*`)
- [ ] Chore / tooling (`chore/*`)

## How was this tested?

<!-- Commands run, and what you observed. -->

```
...
3:## 2026-07-27 — Vikram
58:## 2026-07-30 — Kanika
86:## 2026-07-30 — Vikram
```

Figure 7 — Version-management assets: CI workflows, PR template, CHANGELOG

2.2 System building

PreMerge is packaged with **setuptools** (`pyproject.toml`), installed with `pip install -e .`, built into a wheel with `python -m build`, and can be containerised with the provided `Dockerfile`. GitHub Actions (`tests.yml`) builds and tests on every push and PR across Python 3.11 and 3.12.

System building: pip install + wheel

```
$ rm -rf /tmp/dist; pip install -e . 2>&1 | grep -v WARNING | tail -4; python -m build --wheel -o /tmp/dist . 2>&1 | tail -4; ls -l /tmp/dist; python --version
    Uninstalling premerge-0.1.0:
      Successfully uninstalled premerge-0.1.0
Successfully installed premerge-0.1.0

adding 'premerge-0.1.0.dist-info/top_level.txt'
adding 'premerge-0.1.0.dist-info/RECORD'
removing build/bdist.linux-x86_64/wheel
Successfully built premerge-0.1.0-py3-none-any.whl
total 12
-rw-r--r-- 1 root root 9589 Oct 16 15:47 premerge-0.1.0-py3-none-any.whl
Python 3.13.16
```

Figure 8 — Editable install and wheel build (premerge-0.1.0-py3-none-any.whl)
```

System building: Dockerfile + CI workflows

```
$ cat Dockerfile; echo ------; head -40 .github/workflows/tests.yml; echo ------; head -22 pyproject.toml
FROM python:3.13-slim

RUN apt-get update \
    && apt-get install -y --no-install-recommends git \
    && rm -rf /var/lib/apt/lists/* \
    && git config --system --add safe.directory '*'

WORKDIR /app
COPY . .
RUN pip install --no-cache-dir .

WORKDIR /repo
ENTRYPOINT ["premerge"]
CMD ["--help"]
------
name: Tests

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  pytest:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        python-version: ["3.11", "3.12"]

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}

      - name: Configure git
        # The test fixtures build synthetic repos with real `git` calls, which
        # refuse to commit without an identity configured.
        run: |
          git config --global user.email "ci@example.com"
          git config --global user.name "CI"
          git config --global init.defaultBranch main

      - name: Install
        run: |
          python -m pip install --upgrade pip
          pip install -e .
          pip install pytest pytest-cov

      - name: Run tests
        run: pytest -v --cov=premerge --cov-report=term-missing
------
[project]
name = "premerge"
version = "0.1.0"
description = "Predicts merge conflicts before they happen by analyzing git history coupling and live branch divergence."
requires-python = ">=3.11"
dependencies = [
    "click>=8.1",
    "rich>=13.0",
]

[project.scripts]
premerge = "premerge.cli:main"

[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[tool.setuptools.packages.find]
include = ["premerge*"]

[tool.pytest.ini_options]
testpaths = ["tests"]
```

Figure 9 — Build configuration: Dockerfile, CI workflow and pyproject.toml

> The Docker image could not be built in the sandbox used to prepare this pack (no Docker daemon). The `Dockerfile` is shown above; run `docker build -t premerge .` on your machine and add that screenshot here before submission.

3. Screenshots of Developed Functionalities

Every feature is demonstrated on a reproducible demo repository created by `docs/review3/demo.sh`: history in which `auth.py` and `session.py` always change together; `feature-a/b/c` edit the same lines of `auth.py`; `feature-d` edits only `session.py`; `feature-e` edits an unrelated file.

3.1 CLI and built-in help

premerge --help

```
$ premerge --help; echo; premerge analyze --help
Usage: premerge [OPTIONS] COMMAND [ARGS]...

  PreMerge: predict merge conflicts before they happen.

Options:
  --help  Show this message and exit.

Commands:
  analyze  Rank branch pairs by merge-conflict risk.

Usage: premerge analyze [OPTIONS] [BRANCHES]...

  Rank branch pairs by merge-conflict risk.

  With two BRANCHES given, analyzes just that pair. With one BRANCH given,
  analyzes it against --base. With none given and --base set, analyzes every
  other local branch against --base; with neither given, analyzes every
  pairwise combination of local branches (the useful default for "which of my
  branches are about to collide with each other").

Options:
  --repo TEXT    Path to the git repository.
  --base TEXT    Base branch to compare against (default: current branch).
  --json         Output machine-readable JSON instead of a table.
  --config TEXT  Path to premerge.config.json.
  --no-cache     Skip the historical-coupling cache.
  --help         Show this message and exit.
```

Figure 10 — Built-in CLI help (US-26)

3.2 All-pairs ranked analysis

All-pairs analysis (ranked)

```
$ premerge analyze
```

PreMerge Conflict Risk Report

Branch A	Branch B	Risk	Why
feature-a	feature-b	0.80	auth.py: both branches edit overlapping lines
feature-a	feature-c	0.80	auth.py: both branches edit overlapping lines
feature-b	feature-c	0.80	auth.py: both branches edit overlapping lines
feature-a	feature-d	0.20	auth.py <-> session.py: historically change together (coupling=1.00) — one branch touched auth.py, the other touched session.py
feature-b	feature-d	0.20	auth.py <-> session.py: historically change together (coupling=1.00) — one branch touched auth.py, the other touched session.py
feature-c	feature-d	0.20	auth.py <-> session.py: historically change together (coupling=1.00) — one branch touched auth.py, the other touched session.py
feature-a	feature-e	0.00	no overlap or coupling detected
feature-a	main	0.00	no overlap or coupling detected
feature-b	feature-e	0.00	no overlap or coupling detected
feature-b	main	0.00	no overlap or coupling detected
feature-c	feature-e	0.00	no overlap or coupling detected
feature-c	main	0.00	no overlap or coupling detected
feature-d	feature-e	0.00	no overlap or coupling detected
feature-d	main	0.00	no overlap or coupling detected
feature-e	main	0.00	no overlap or coupling detected

Figure 11 — All branch pairs ranked by risk, colour-coded red/yellow/green (US-4, 16, 17)

3.3 Single pair — direct line overlap

Branch pair: direct line overlap

```
$ premerge analyze feature-a feature-b
```

PreMerge Conflict Risk Report

Branch A	Branch B	Risk	Why
feature-a	feature-b	0.80	auth.py: both branches edit overlapping lines

Figure 12 — Line-level overlap → risk 0.80 (US-3, 5, 6, 7)

3.4 Single pair — hidden historical coupling

Branch pair: hidden coupling

```
$ premerge analyze feature-c feature-d
```

PreMerge Conflict Risk Report

Branch A	Branch B	Risk	Why
feature-c	feature-d	0.20	auth.py <-> session.py: historically change together (coupling=1.00) — one branch touched auth.py, the other touched session.py

Figure 13 — No shared file, yet coupling 1.00 → risk 0.20: the hidden-conflict signal (US-8–11, 14)

3.5 Base-branch mode

Base mode

```
$ premerge analyze --base feature-a
```

PreMerge Conflict Risk Report

Branch A	Branch B	Risk	Why
feature-a	feature-b	0.80	auth.py: both branches edit overlapping lines
feature-a	feature-c	0.80	auth.py: both branches edit overlapping lines
feature-a	feature-d	0.20	auth.py <-> session.py: historically change together (coupling=1.00) — one branch touched auth.py, the other touched session.py
feature-a	feature-e	0.00	no overlap or coupling detected
feature-a	main	0.00	no overlap or coupling detected

Figure 14 — One branch vs. all others (US-25)

3.6 JSON output

JSON output

```
$ premerge analyze feature-a feature-b --json
[
  {
    "branch_a": "feature-a",
    "branch_b": "feature-b",
    "score": 0.8,
    "file_overlap": [
      "auth.py"
    ],
    "line_overlap_files": [
      "auth.py"
    ],
    "coupled_pairs": [],
    "explanations": [
      "auth.py: both branches edit overlapping lines"
    ]
  }
]
```

Figure 15 — Machine-readable JSON consumed by the GitHub Action (US-15)

3.7 Configurable weights and SQLite cache

Config weights + cache

```
$ echo '--- default weights'; premerge analyze feature-c feature-d --json | grep '"score"'; echo '--- custom weights (coupling=0.6)'; premerge analyze feature-...
--- default weights
    "score": 0.2,
--- custom weights (coupling=0.6)
    "score": 0.6,
--- cache file created
-rw-r--r-- 1 root root 12288 Oct 16 15:43 .premerge_cache.sqlite
--- cold vs cached output identical?
5f1c7482456a5b84138409f34222048b  -
5f1c7482456a5b84138409f34222048b  -
5f1c7482456a5b84138409f34222048b  -
```

Figure 16 — Tunable weights change the score; SQLite cache yields identical results (US-12, 18, 19)

3.8 Validation against real git

Git vs PreMerge

```
$ premerge analyze feature-a feature-b | head -8; echo; echo '--- git merge feature-a + feature-b:'; git checkout -q feature-a; git merge --no-commit feature-b...
```

PreMerge Conflict Risk Report

Branch A	Branch B	Risk	Why
feature-a	feature-b	0.80	auth.py: both branches edit overlapping lines

```
--- git merge feature-a + feature-b:
Auto-merging auth.py
CONFLICT (content): Merge conflict in auth.py
Automatic merge failed; fix conflicts and then commit the result.

--- feature-c + feature-d: PreMerge risk 0.20 (hidden coupling); git merge says:
Automatic merge went well; stopped before committing as requested
```

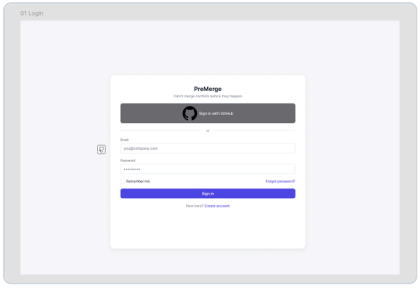
Figure 17 — PreMerge's 0.80 prediction matches a real `git merge` conflict; the 0.20 pair merges textually clean but is flagged as logically coupled

3.9 GitHub Action (CI integration)

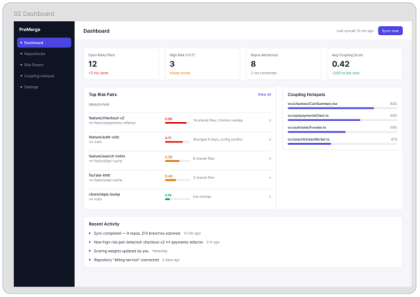
On every `pull_request` event the composite action (`github_action/action.yml`) installs PreMerge, runs `premerge analyze origin/<base> origin/<head> --json` and `post_comment.py` posts a single comment identified by the marker `<!-- premerge-report -->`, updating it in place on later pushes (US-21–23). The workflow is dogfooded on this repository.

3.10 UI design (Figma prototype of the proposed dashboard)

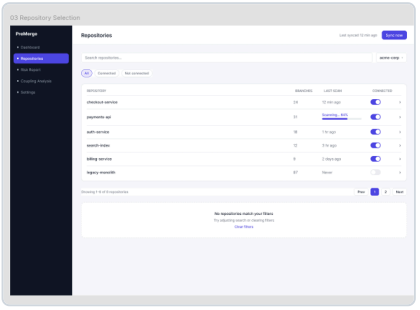
The planned web dashboard (MoSCoW item C1) is designed in Figma; six screens are shown below.



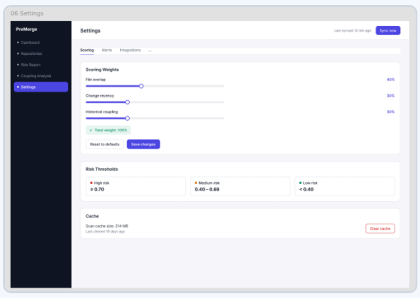
Login



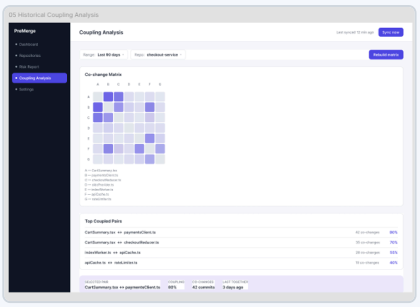
Dashboard



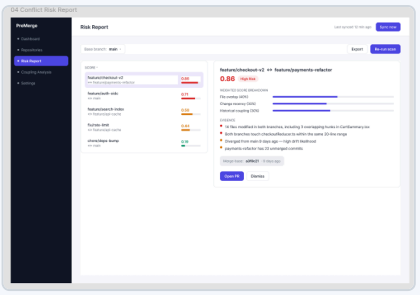
Repository selection



Conflict risk report



Coupling analysis



Settings

4. Tools and Technologies Used

Category	Tool / Technology	Purpose in PreMerge
Language	Python 3.11+ (developed on 3.13)	All application code; modern typing, dataclasses
CLI framework	click ≥ 8.1	Command/option parsing, built-in help
Terminal UI	rich ≥ 13	Colour-coded risk table
VCS engine	git CLI via `subprocess`	log, diff -U0, merge-base, rev-parse (no GitPython)
Storage	SQLite (stdlib `sqlite3`)	Coupling cache keyed by HEAD commit
Data format	JSON (stdlib)	Config file and machine-readable output
CI integration	GitHub Actions (composite action), stdlib `urllib`	PR comment posting via GitHub REST API
Testing	pytest, pytest-cov, click CliRunner	Unit, integration, regression tests and coverage
Mutation testing	mutmut 3.8	Test-suite quality measurement
Packaging / build	setuptools, pip, `build`	Installable package and wheel
Containerisation	Docker (python:3.13-slim)	Zero-setup execution
Version control	Git, GitHub, GitHub Flow, PR template	Collaboration, review, branch protection
Design	Figma, draw.io, hand-built SVG	UI prototype, architecture, DFD/ERD/UML
Project management	GitHub Issues & Projects, MoSCoW	25 tracked user stories